

# **Parking Lot Management System**

## **1. Problem Statement and Problem Formulation**

What the problem is and how we'll define it

In busy places like shopping malls and office buildings, keeping track of parking by hand can be tough. With so many cars coming and going, parking staff struggle to remember which spots are taken, which car is where, and how much each vehicle still owes.

We built a Parking Lot Management System in Python to tackle this issue. The system lets parking staff manage vehicles easily and in an orderly way.

The user can enter the vehicle number and pick the vehicle type. The system then finds an open spot and assigns the nearest one. It also records when the vehicle enters. When it leaves, the system figures out how long it was parked and charges accordingly.

The system tracks both taken and open parking spots, keeping vehicle info and transaction records handy for the final report.

This project makes parking easier and cuts down on mistakes that happen when things are done by hand.

## **2. Objective and Expected Outcomes**

Objective: Build a simple parking management system using Python.

Scan and register vehicles as they arrive.

Assign an open parking spot automatically.

Track when vehicles arrive and leave.

Parking fees vary based on vehicle type and how long you're parked.

Use Python data structures to keep track of parking info.

Save parking transactions to a file.

To let people see which spots are free and which are taken.

To generate a simple daily parking report.

When the project is done, the system should be able to:

Successfully registering a vehicle.

Assign the nearest open slot.

Show which slots are filled and which are open.

Determine the parking fee.

Release the slot as soon as the vehicle leaves.

Write down the transaction details.

Display the total parking revenue.

Handle wrong inputs without stopping the program.

### **3. Requirements, Constraints, and Assumptions**

You'll need Python 3 to build this project.

- A computer or laptop
- A Python IDE like Python IDLE or VS Code
- We're using a text file to store parking records, but the current system has limits — the number of spots is fixed.

We're only looking at three types of vehicles.

A vehicle can't be registered twice while it's already parked.

If every parking spot's taken, the vehicle won't be allowed in.

You can't leave unless the vehicle is registered.

#### **Assumptions**

For this project, we're assuming each vehicle has its own unique number.

The parking spots are already numbered.

The vehicle gets the next open spot.

Parking costs vary depending on the kind of vehicle.

A parking attendant uses the console to run the system.

#### **4. What I learned in the Python course**

This project gave us a chance to put many of the Python ideas from class into practice.

Variables: store key data like vehicle numbers, types, parking spots, entry/exit times, and fees

Conditional Statements: If, elif, and else help check things like whether a slot is free or if a vehicle even exists.

Loops: The main menu keeps running in a loop until the user picks exit. We loop through each parking slot to find what we're looking for.

Functions: Functions help break the program into smaller, manageable pieces. For instance, each task—like registering a vehicle, assigning a spot, processing exit, and creating the report—is handled by its own separate function.

Strings: String operations help read and check vehicle numbers.

Lists: We use a list to keep track of the parking slot numbers.

Dictionaries: We use a dictionary to keep track of parked vehicles, including their type, assigned slot, and entry time.

Sets: We use a set to track which slots are currently taken.

Tuples: Tuples hold the fixed tariff amounts for each vehicle type.

File Handling: Entry and exit transactions are saved in a text file.

Modules: The billing calculation is handled in its own Python file, `billing.py`, and imported into the main program.

#### **5. Methodology**

The Parking Lot Management System is developed in Python by using a simple menu-driven method. It is designed so that the processes of allowing vehicles to enter, allocating a parking slot, enabling vehicles to exit, carrying out billing and generating reports can all be more easily managed.

##### **Step 1: Parking Slot Setup**

Parking spaces are provided on three levels: the Ground, the Podium, and the Terrace.

Each slot is stored in a dictionary together with its floor and the type of vehicle that is suitable.

A set is employed in order to keep record of the slots that are currently occupied.

#### Step 2: Vehicle Registration

The user inputs the vehicle registration number.

The system verifies if the vehicle number is valid.

It also verifies if the vehicle is already parked.

The user then chooses the type of vehicle: either Two-Wheeler, Car, or Heavy Vehicle.

#### Step 3: Slot Allocation

The system looks for a free slot that is suitable.

The vehicle is given the closest slot that is available.

When the slot has been assigned, it is then regarded as being occupied.

The vehicle number, the type of vehicle, the slot number, and the time of entry are stored.

#### Step 4: Parking Transaction Logging

Each entry for a vehicle is recorded in a parking log file.

On restarting the program, the previous log is read in order to restore the vehicles which are still parked.

This enables the system to resume from the point at which it was previously stopped. Step 5: Vehicle Exit and Billing

To leave a vehicle, the user has to enter the registration number.

The system verifies if the vehicle is currently parked.

The length of time for which the car is parked is determined by the time it enters and the time it leaves.

The amount you have to pay for parking depends on the type of vehicle and how long you park.

Once the payment has been calculated, the slot is released and is then marked as available.

The log file contains the details of the exit transaction and the fare.

#### Step 6: Parking Status

The system shows the present parking situation.

It gives the number of occupied slots, total slots, and free slots for each type of vehicle.

This enables the operator to immediately grasp the amount of available parking space.

#### Step 7: Tariff Management

The system has a tariff card which varies according to the type of vehicle.

The billing module works out the amount to be charged in accordance with the relevant parking rates.

#### Step 8: Report Generation

The log file contains all the parking transactions that have been stored.

The system determines the number of entries, exits, and the amount of revenue.

A report at the end of the day is produced using the transaction data and the up-to-date parking information.

#### Step 9: Error Handling

\* The system handles common errors such as:

- \* Invalid vehicle number

- \* Duplicate vehicle entry

- \* Vehicle not found

- \* Parking lot full

- \* Invalid vehicle type

- \* Invalid menu choice

Messages that are appropriate are shown to the user so that they can make corrections to the input.

#### Step 10: Program Termination

The system will keep showing the menu until the user chooses **\*\*Quit\*\***.

When Quit is chosen, the current session is ended and the program stops.

### Simple Flow

Car arrives → registers → finds available spot → gets assigned a parking space → entry logged → car departs → time calculated → bill generated → spot freed → record updated

## 6. PSEUDOCODE :

BEGIN

SET LOG\_PATH = "parking\_log.txt"

SET VEHICLE\_LABEL

TW → "Two-Wheeler"

CAR → "Car"

HV → "Heavy Vehicle"

FUNCTION rebuild\_state()

CALL build\_slot\_pool()

GET slot\_pool and occupied\_slots

CALL reload\_open\_sessions(LOG\_PATH)

GET open\_sessions

CREATE vehicle\_records

FOR each vehicle in open\_sessions DO

GET slot\_id

GET vehicle\_type

GET entry\_time

MARK slot\_id as occupied

STORE slot\_id, vehicle\_type and entry\_time

```

        in vehicle_records

    END FOR

    RETURN slot_pool, occupied_slots, vehicle_records

END FUNCTION

FUNCTION prompt_vehicle_type()

    DISPLAY "1. Two-Wheeler"

    DISPLAY "2. Car"

    DISPLAY "3. Heavy Vehicle"

    INPUT choice

    IF choice = 1 THEN

        RETURN "TW"

    ELSE IF choice = 2 THEN

        RETURN "CAR"

    ELSE IF choice = 3 THEN

        RETURN "HV"

    ELSE

        RETURN INVALID

    END IF

END FUNCTION

FUNCTION handle_entry()

    INPUT vehicle_number

    TRY

        CALL normalise_vehicle_number(vehicle_number)

```

```
IF vehicle number is invalid THEN

    DISPLAY "REJECTED - Invalid vehicle number"

    RETURN

END IF

IF vehicle_number already exists in vehicle_records THEN

    DISPLAY "REJECTED - Vehicle is already parked"

    RETURN

END IF

CALL prompt_vehicle_type()

GET vehicle_type

IF vehicle_type is INVALID THEN

    DISPLAY "REJECTED - Invalid category choice"

    RETURN

END IF

TRY

    CALL find_nearest_slot(slot_pool, occupied_slots, vehicle_type)

    GET slot_id

IF parking lot is full THEN

    DISPLAY "REJECTED - Parking lot is full"

    RETURN

END IF

MARK slot_id as occupied

SET entry_time = current date and time

STORE vehicle_number, slot_id,
```

vehicle\_type and entry\_time

CALL append\_event(

LOG\_PATH,

"ENTRY",

vehicle\_number,

vehicle\_type,

slot\_id

)

DISPLAY vehicle number

DISPLAY vehicle type

DISPLAY allocated slot

DISPLAY entry time

END FUNCTION

FUNCTION handle\_exit()

INPUT vehicle\_number

TRY

CALL normalise\_vehicle\_number(vehicle\_number)

IF vehicle number is invalid THEN

DISPLAY "REJECTED - Invalid vehicle number"

RETURN

END IF

IF vehicle\_number is NOT in vehicle\_records THEN

DISPLAY "REJECTED - Vehicle is not parked"

```
    RETURN

END IF

GET vehicle record

SET exit_time = current date and time

CALCULATE parking_duration

    = exit_time - entry_time

IF parking_duration is less than 1 minute THEN

    SET parking_duration = 1 minute

END IF

CALL calculate_charge(

    vehicle_type,

    parking_duration

)

GET fare

MARK vehicle's slot as FREE

REMOVE vehicle from vehicle_records

CALL append_event(

    LOG_PATH,

    "EXIT",

    vehicle_number,

    vehicle_type,

    slot_id,

    fare

)
```

DISPLAY vehicle number

DISPLAY slot number

DISPLAY parking duration

DISPLAY total fare

END FUNCTION

FUNCTION handle\_status()

DISPLAY "Type    Occupied    Total    Free"

CALL occupancy\_snapshot(

    slot\_pool,

    occupied\_slots

)

FOR each vehicle\_type DO

    GET occupied\_count

    GET total\_count

    CALCULATE free\_count

        = total\_count - occupied\_count

    DISPLAY vehicle type

    DISPLAY occupied\_count

    DISPLAY total\_count

    DISPLAY free\_count

END FOR

END FUNCTION

FUNCTION handle\_report()

    CALL read\_all\_events(LOG\_PATH)

```

GET all parking events

CALL rebuild_state()

CALL generate_report(
    events,
    slot_pool,
    occupied_slots
)

DISPLAY generated report

END FUNCTION

FUNCTION main()

    CALL rebuild_state()

    DISPLAY "Bay-Watch Complex parking console started"

    IF vehicles are restored THEN

        DISPLAY number of restored vehicles

    END IF

    WHILE TRUE DO

        DISPLAY "===== PARKING CONSOLE ====="

        DISPLAY "1. Vehicle Entry"

        DISPLAY "2. Vehicle Exit & Billing"

        DISPLAY "3. Live Slot Status"

        DISPLAY "4. View Tariff Card"

        DISPLAY "5. End-of-Day Report"

        DISPLAY "6. Quit"

        INPUT choice

```

```
IF choice = 1 THEN
    CALL handle_entry()
ELSE IF choice = 2 THEN
    CALL handle_exit()
ELSE IF choice = 3 THEN
    CALL handle_status()
ELSE IF choice = 4 THEN
    CALL tariff_card()
    DISPLAY tariff details
ELSE IF choice = 5 THEN
    CALL handle_report()
ELSE IF choice = 6 THEN
    DISPLAY "Shift closed. Goodbye."
    BREAK
ELSE
    DISPLAY "Please choose a valid option (1-6)."
END IF

END WHILE

END FUNCTION

CALL main()

END
```

## 7. Source code:

"""

main.py

Bay-Watch Complex -- Multi-Level Parking Console

A menu-driven front end. All the real logic lives inside the

`parking\_system` package (slot\_manager, billing, transaction\_log,

reports, utils, exceptions) -- this file only wires the menu options

to those functions and formats what the operator sees.

Run: python main.py

Log: parking\_log.txt is created next to this file on first entry.

"""

from datetime import datetime

from parking\_system import slot\_manager, billing, transaction\_log, reports

from parking\_system.utils import normalise\_vehicle\_number

from parking\_system.exceptions import (

LotFullError, UnknownVehicleTypeError, InvalidVehicleNumberError,

VehicleNotParkedError, DuplicateEntryError,

)

LOG\_PATH = "parking\_log.txt"

VEHICLE\_LABEL = {"TW": "Two-Wheeler", "CAR": "Car", "HV": "Heavy Vehicle"}

def rebuild\_state():

"""Fresh slot pool + replay the log so the console remembers who

was still parked the last time it was closed."""

```

slot_pool, occupied = slot_manager.build_slot_pool()

open_sessions = transaction_log.reload_open_sessions(LOG_PATH)

vehicle_records = {}

for vehicle_no, row in open_sessions.items():

    slot_manager.occupy(slot_pool, occupied, row["slot_id"])

    vehicle_records[vehicle_no] = {

        "slot_id": row["slot_id"],

        "vtype": row["vehicle_type"],

        "entry_time": datetime.fromisoformat(row["timestamp"]),

    }

return slot_pool, occupied, vehicle_records

def prompt_vehicle_type():

    print("Vehicle type -> [1] Two-Wheeler [2] Car [3] Heavy Vehicle")

    choice = input("Choose 1/2/3: ").strip()

    return {"1": "TW", "2": "CAR", "3": "HV"}.get(choice)

def handle_entry(slot_pool, occupied, vehicle_records):

    raw_number = input("Enter vehicle registration number: ")

    try:

        vehicle_no = normalise_vehicle_number(raw_number)

    except InvalidVehicleNumberError as err:

        print(f"[REJECTED] {err}")

        return

    if vehicle_no in vehicle_records:

        print(f"[REJECTED] {vehicle_no} is already logged as parked (no exit recorded).")

```

```

        return

vtype = prompt_vehicle_type()

if vtype is None:

    print("[REJECTED] Invalid category choice.")

    return

try:

    slot_id = slot_manager.find_nearest_slot(slot_pool, occupied, vtype)

except (LotFullError, UnknownVehicleTypeError) as err:

    print(f"[REJECTED] {err}")

    return

slot_manager.occupy(slot_pool, occupied, slot_id)

entry_time = datetime.now()

vehicle_records[vehicle_no] = {"slot_id": slot_id, "vtype": vtype, "entry_time":
entry_time}

transaction_log.append_event(LOG_PATH, "ENTRY", vehicle_no, vtype, slot_id)

print(f"[OK] {vehicle_no} ({VEHICLE_LABEL[vtype]}) assigned to slot {slot_id} at
{entry_time:%H:%M:%S}")

def handle_exit(slot_pool, occupied, vehicle_records):

    raw_number = input("Enter vehicle registration number leaving: ")

    try:

        vehicle_no = normalise_vehicle_number(raw_number)

    except InvalidVehicleNumberError as err:

        print(f"[REJECTED] {err}")

        return

    if vehicle_no not in vehicle_records:

```

```

    print(f'[REJECTED] {VehicleNotParkedError(vehicle_no)}')

    return

record = vehicle_records.pop(vehicle_no)

exit_time = datetime.now()

duration_minutes = max(1, (exit_time - record["entry_time"]).total_seconds() / 60)

fare = billing.calculate_charge(record["vtype"], duration_minutes)

slot_manager.vacate(occupied, record["slot_id"])

transaction_log.append_event(

    LOG_PATH, "EXIT", vehicle_no, record["vtype"], record["slot_id"], amount=fare

)

print(f'[OK] {vehicle_no} left slot {record["slot_id"]} after {duration_minutes:.1f} min ->
Rs.{fare:.2f}')

def handle_status(slot_pool, occupied):

    print(f'\n{'Type':14} {'Occupied':>10} {'Total':>8} {'Free':>8}")

    for vtype, (used, total) in slot_manager.occupancy_snapshot(slot_pool, occupied).items():

        print(f'{VEHICLE_LABEL[vtype]:14} {'used':>10} {'total':>8} {'total - used':>8}")

def handle_report():

    events = transaction_log.read_all_events(LOG_PATH)

    slot_pool, occupied, _ = rebuild_state()

    print("\n" + reports.generate_report(events, slot_pool, occupied))

MENU = ""

```

=== Bay-Watch Complex -- Parking Console ===

1. Vehicle Entry

2. Vehicle Exit & Billing

3. Live Slot Status

4. View Tariff Card

5. End-of-Day Report

6. Quit

"""

def main():

    slot\_pool, occupied, vehicle\_records = rebuild\_state()

    print("Bay-Watch Complex parking console started.")

    if vehicle\_records:

        print(f"({len(vehicle\_records)} vehicle(s) restored from a previous session.)")

    while True:

        print(MENU)

        choice = input("Select an option: ").strip()

        if choice == "1":

            handle\_entry(slot\_pool, occupied, vehicle\_records)

        elif choice == "2":

            handle\_exit(slot\_pool, occupied, vehicle\_records)

        elif choice == "3":

            handle\_status(slot\_pool, occupied)

        elif choice == "4":

            print("\n" + billing.tariff\_card())

        elif choice == "5":

            handle\_report()

        elif choice == "6":

            print("Shift closed. Goodbye.")

```
        break

    else:

        print("Please choose a valid option (1-6).")

if __name__ == "__main__":

    main()
```

## **8. Test Cases and Anticipated Results**

### **Test Case 1 - Valid Vehicle Entry**

Test:

Choose option `1` and input a valid vehicle number for example `TN01AB1234`. Press `2` for Car.

Expected Outcome:

The vehicle number should be accepted and allocated the nearest available slot. Output the vehicle number, vehicle type, slot number and entry time.

### **Test Case 2 - Invalid Vehicle Number**

Test:

Choose option `1` and type in a "non valid" vehicle number such as `12345`.

Anticipated Outcome:

It should not take the vehicle number and should show an error that vehicle number is invalid.

### **Test Case 3 – Duplicate Vehicle Entry**

Test:

Attempt to enter a vehicle that already exists in the parking lot.

Expected Result:

Should not allow entry and say vehicle is already Logged as Parked.

#### Test Case 4 - Invalid Vehicle Type

Test:

When asked to Enter Vehicle Type, enter a value other than 1,2 or 3.

Expected Result:

It should give invalid choice in category."

#### Test Case 5 – Valid Vehicle Exit

Test:

Choose option 2 and provide the registration number of a vehicle that is parked. Expected Result: The system should search for the vehicle, calculate parking time and charge, free the occupied slot, write EXIT record in parking\_log.txt, and print the vehicle no., slot no., parking time, and final charge.

#### Test Case 6 – Vehicle Not Parked

Test:

Choose option 2 and enter a vehicle number that is valid but not parked.

Expected Result:

A rejection message should be displayed stating that the vehicle is not parked.

#### Test Case 7 – Invalid Vehicle Number During Exit

Test:

Select option 2 and enter a fake vehicle reg number.

Expected Result:

The input should be rejected and the error message "Invalid vehicle number" should be displayed.

#### Test Case 8 – Live Slot Status

Test:

Choose option 3.

Expected Result:

The system should show the number of occupied, total, and free slots for every vehicle type.

#### Test Case 9 – View Tariff Card

Test:

Select option 4.

Expected Result:

The user should be able to see tariff details for Two-Wheeler, Car and HV.

#### Test Case 10 – End-of-Day Report

Test:

Choose option 5.

Expected Result:

The system should read the parking log, and then show the parking and sales report.

#### Test Case 11 – Previous Session Recovery

Test:

Launch the application when the parking log has vehicles entered but no exit record for a few of them.

Expected Result: It should resurrect those vehicles and take their parking slots. Test Case 12 – Invalid Menu Choice

Test: Type any option that is not 1 to 6, say 9.

Expected Result: System should print "Please select a valid option (1-6)."

## 9.Outputs:

```
=== Bay-Watch Complex -- Parking Console ===
1. Vehicle Entry
2. Vehicle Exit & Billing
3. Live Slot Status
4. View Tariff Card
5. End-of-Day Report
6. Quit

Select an option: 1
Enter vehicle registration number: TN08AB1456
Vehicle type -> [1] Two-Wheeler [2] Car [3] Heavy Vehicle
```

```
Select an option: 1
Enter vehicle registration number: TN08AB1456
Vehicle type -> [1] Two-Wheeler [2] Car [3] Heavy Vehicle
Choose 1/2/3: 1
[OK] TN08AB1456 (Two-Wheeler) assigned to slot GRO-TW-01 at 09:28:32
```

```
Select an option: 2
Enter vehicle registration number leaving: TN08AB1456
[OK] TN08AB1456 left slot GRO-TW-01 after 1.0 min -> Rs.0.00
```

```
Select an option: 3
```

| Type          | Occupied | Total | Free |
|---------------|----------|-------|------|
| Two-Wheeler   | 0        | 10    | 10   |
| Car           | 1        | 20    | 19   |
| Heavy Vehicle | 0        | 5     | 5    |

Select an option: 4

| Vehicle | First Hr | Extra Hr | Daily Cap |
|---------|----------|----------|-----------|
| TW      | Rs.10    | Rs.5     | Rs.80     |
| CAR     | Rs.20    | Rs.15    | Rs.150    |
| HV      | Rs.40    | Rs.25    | Rs.250    |

=====

BAY-WATCH COMPLEX -- SHIFT REPORT (02-Sep-2026 09:31)

=====

| Type | Occupied/Total | Exits | Revenue |
|------|----------------|-------|---------|
| TW   | 0/10           | 1     | Rs.0.00 |
| CAR  | 1/20           | 0     | Rs.0.00 |
| HV   | 0/5            | 0     | Rs.0.00 |

-----

Total vehicles registered this log : 2

Total revenue collected : Rs.0.00

=====

Select an option: 6  
Shift closed. Goodbye.

## 10. Results and Validation

The Console Menu-driven Screen for the Parking Lot Management System was tested by executing various tasks. The results demonstrate that the major functions of the system realize the expected function.

### 1 Vehicle Entry Result

The vehicle registration number "TN08AB1456" was submitted. The Type of Vehicle is Two-Wheeler. the vehicle was duly accepted by the system and given the parking slot "GR0-TW-01". The entry time was also entered as 09:28:32.

Output: Slot allocation and Vehicle entry got executed properly.

## 2 Vehicle Exit and Billing Result

The exist vehicle becd name is TN08AB1456 (the same vehicle we used in above example). The vehicle was located and parked time calculated by the system. The vehicle remained for 1.0 minute , therefore the calculated parking fee Rs. 0.00. Once the car just left, the system will also release the parking slot.

Output: Vehicle exit, duration fee calculation, billing and slot release were executed successfully.

## 3 Live Slot Status Result

Now, after testing the Live Slot Status option after the vehicle exit. The system gave the following information:

\* Two Wheeler : 0 out of 10 occupied, 10 free

\* Car : 1 out of 20 occupied, 19 free

\* Heavy Vehicle : 0 out of 5 occupied, 5 free

This confirms the even when one goes in and out with a vehicle, the System is updating properly, The available Parking Spaces updating number.

Result: Live slot tracking and available calculation was performed correctly.

## 4 Results of Tariff Card

There were successful testing of the View Tariff Card.

The system showed : Two-Wheeler: First Hour – Rs.10, Above that Hour – Rs.5, Daily Cap – Rs.80

Car: First Hour – Rs.20, Additional Hour – Rs.15, Daily Cap – Rs.150

Heavy Vehicle: First Hour – Rs.40, Again the Second Hour for Rs.25, rack up to daily cap of Rs 250!

Result: The tariff details were correctly displayed by the system.

## 5 End-of-Day Report Result

End-of-Day Report option was tested after vehicle transaction

The report shows:

Two-Wheeler: 0/10 full, 1 exit, Rs.0.00 revenue

Car: 1/20 full, 0 exits, Rs.0.00 revenue

Heavy Vehicle: 0/5 full, 0 exits, Rs.0.00 revenue

Number of vehicles recorded in the log: 2

Total revenues collected: Rs.0.00

The report found one Two-Wheeler had exited while a car was still there.

Result: From the logs kept, the system was able to produce parking status, exit count and revenue report.

#### 6 End-of-Day Report Result

The End-of-Day Report function was verified post the vehicle transactions.

The report shows:

Two-Wheeler: 0/10 full, 1 exit, Rs.0.00 revenue

Car: 1/20 full, 0 exits, Rs.0.00 revenue

Heavy Vehicle: 0/5 full, 0 exits, Rs.0.00 revenue

Number of vehicles registered in log: 2 .

Total revenues collected: Rs.0.00

Result: The system generated a report with the parking status, no of exits and revenue from the stored log records.

#### 7 Overall Validation

The test case passes, which means that all the significant features of the Parking Lot Management System are working fine. Successful run for Vehicle Entry, slot allocation, Vehicle exit, Billing, Live parking, Tariff display, Report generation, Log based tracking and Program termination has been evaluated through the console.

The outputs also indicate that the parking status is updated when a car departs. The revenue calculated was Rs.0.00 as the Two-Wheeler tested stayed for 1 minute, which was reflected properly in the report.

## **11. Evaluation, Comparison, Trade-offs and Rationale of the Final Protocol**

### **Analysis**

The newly developed Parking Lot Management System is reviewed in term of its vehicle entry, vehicle exit, billing, live slot status, tariff display, report generation and slot allocation.

The system employs different modules for different processes, which allows for easier understanding and maintenance of the program. The gliding parking slots are managed, while the vehicle information is saved during the present parking time. The entry and exit information is to be saved in the transaction log.

In testing, the vehicle TN08AB1456 was registered successfully as Two-Wheeler and allotted slot GRO-TW-01. When the vehicle left the system after 1.0 minute the fare was calculated as Rs.0.00\*K and the slot was released. The real-time reports and end of day report also properly reflected the occurrence of parking changes in the above two cases.

### **Comparison**

The system is up against a traditional parking management technique.

#### **Manual method:**

- \* The details of the vehicles are written in registers or manually kept.
- \* It also takes more time to hunt for an empty slot.
- \* There could be errors in calculation of parking fee by manual mode.
- \* Old parking information not easy to trace.
- \* There is more manual work involved in the daily reporting.

#### **Developed system:**

- \* The vehicle information is processed electronically. \* It chooses the closest appropriate place.
- \* The billing system figures parking fees.

- \* All the entries and exits are logged to a file.
- \* You can verify parking immediately!
- \* You can produce reports from the transaction logs that are saved.

Hence, the proposed system gives a professional and better way to track your parking rather than doing it manually.

### Trade-offs

The system was simple enough for a Python project, yet complete. The CheapNewsSystem was suitable for use in a Python project. Some decisions have advantages and disadvantages.

Console interface: Developing a console interface is a straightforward process, however, it is not as user-friendly as the graphical version.

Logfile storage: It is straightforward to implement and is good for storing transactions, however it lacks the power of a database.

Modular Design Separating slot management, billing, logging and is easier to maintain but you need several modules for this task.

Auto slot assignment: It decreases manual workload and enables an effective use of available spaces, although it depends on predefined slot configuration.

Automatic billing: Calculations are made faster, and manual calculation errors are eliminated. Remember that actual digital payment is not included.

### Justification of the Final Solution

The chosen solution is that which meets the basic needs of the parking management system while the effort to implement it is minimum.

The appropriate use of Python functions, dictionaries, sets, tuples, modules, file I/O, and exception handling is demonstrated in these required course concepts. The system also includes features such as automatic slot allocation, vehicle tracking, billing, live status checking, tariff viewing, report generation and more. Based upon the given testing information, the system succeeded in running the major functions and creating the what they wanted. Consequently, the proposed solution is an techniques for adding a database, gui, online booking and digital payment facilities have been discussed.

## 12. SDG Relevance

The Parking Lot Management System is primarily associated with SDG 9, SDG 11 and SDG 12. The project employs basic digital management techniques and Python programming language to enhance the management of parking lots and vehicle flow.

### SDG 9 – Industry, Innovation and Infrastructure

The project contributes to the SDG 9 by introducing a digital system to manage the parking lot infrastructure. Instead of managing the details of the parking manually, the system will use Python, dictionaries, sets, file handling and modules to keep track of parking slots, vehicle entries, exits and billing.

Applicability to the project:

- \* Offers a computerized solution for parking infrastructure management.
  - \* Decreases reliance on manual parking logs.
  - \* Performs slot allocation and billing using Python automation .
  - \* Keeps a log file for parking transactions.
- SDG 11 – Sustainable Cities and Communities

The relevance of the project is directly related to **SDG 11** since effective parking management is one of the methods that organize the vehicle flow within the parking lot. The system indicates free and occupied spaces, which might be of assistance for better management of parking.

### SDG 12 – Responsible Consumption and Production

The project aligns with SDG 12 as it eliminates the need for excess usage of paper in traditional parking logs and recording that information on paper results on transaction information digitally. The system records the details of entry and exit in a log and reports are generated using the stored data.

Applicability to the project:

Decreases the need for paper records.

Keeps data about parkings in digital form.

Monitor parking usage and income.

Authors found that their record management became even more streamlined and effective.

#### Contribution to SDG in general

The contribution of the project in achieving these SDGs is using simple digital solutions in the application for parking management. It promotes efficient use of existing parking assets, facilitates organized urban parking practices and minimizes reliance on manual documentations.

### **13. Conclusion , Limitations and Possible Improvements**

#### **Conclusion**

Parking Lot Management System was developed successfully in Python as a console based application with menu-driven feature. It simplifies vehicle entry, slot allocation, vehicle exiting, billing, tariff details and parking reports .

Based on Python concepts like - functions, dictionaries, sets, modules, file handling, tuples, and exception handling). In addition, it saves parking transactions to a log file, so that when you start up again, you can recover cars that are still parked.

All the key features, including vehicle registration, slot allocation, exit management, billing, live slot status, tariff display and report generation are working fine from the test results.

From the test results, the about section of the vehicle registration, vehicle slot allotment, vehicle unregistration, billing, live available slot, tariff calculator and report the generation of these worked flawlessly. In general, this project offers a very simple and neat management over multi-level parking lot.

#### Constraints/Limitations

Currently the system runs through console interface only and it is without GUI.

Parking slots availability are handled ( added / removed ) according to the following :

The system does not allow online booking.

The payment is computed by the system only and no real transaction is involved.

Instead of a database, the system relies primarily on the parking log file to store transactions.

Notification in real time to owner of vehicles are not available.

## Refinements

A GUI or web-based application can be developed to improve the usability.

A database such as MySQL, SQLite can be used to better store and manage the parking records.

One can add online parking booking as an extension so that they can book the slots beforehand.

Integrate payment systems like upip, card; payment by digital mode.

You may add a “vehicle search” feature to help users to locate a vehicle. Notifications can be enabled for entry, exit and payment confirmation.

The system can be interfaced to sensors/cameras for auto vehicle/slot detection.

Graphs or a dashboard can be added to better illustrate the parking occupancy and revenues.

## **14. Individual Contribution of Group Members**

S. Devadharshini

I have been involved in designing and developing the entire application of Parking Lot Management System. Among other things I did include working on main program flow, menu options to the entry and exit of the vehicles, slot allocation and Module integration. I also participated in testing of application and to verify whether result is as expected.

Soujanya B

My work mainly includes billing, transaction logging, report generation and testing. I also assisted in verifying the parking fees, the entry and exit records and the check for the status of parking and income report. In addition, I assisted in the detection of errors after testing and the confirmation of proper system functioning.

## Team Contribution

Both the team members and are colleagues). We talked about the requirements, assigned chores, tested functions, examined outputs, and refined the program. The project was accomplished using teamwork, collaboration on the code, testing and debugging

## 15. References

1. Fahim, A., Hasan, M., & Chowdhury, M. A. (2021). **Smart parking systems: Comprehensive review based on various aspects.** *Heliyon*, 7(5), e07050. <https://doi.org/10.1016/j.heliyon.2021.e07050>
2. Diaz Ogás, M. G., Fabregat, R., & Aciar, S. (2020). **Survey of Smart Parking Systems.** *Applied Sciences*, 10(11), 3872. <https://doi.org/10.3390/app10113872>
3. Lin, T., Rivano, H., & Le Mouél, F. (2017). **A survey of smart parking solutions.** *IEEE Transactions on Intelligent Transportation Systems*, 18(12), 3229–3253.
4. Al-Turjman, F., & Malekloo, A. (2019). **Smart parking in IoT-enabled cities: A survey.** *Sustainable Cities and Society*, 49, 101608. <https://doi.org/10.1016/j.scs.2019.101608>
5. Idris, M. Y. I., Tamil, E. M., Noor, N. M., Razak, Z., & Fong, K. W. (2009). **Parking guidance system utilizing wireless sensor network and ultrasonic sensor.** *Information Technology Journal*, 8(2), 138–146.
6. Hassoune, K., Dachry, W., Moutaouakkil, F., & Medromi, H. (2016). **Smart parking systems: A survey.** *2016 11th International Conference on Intelligent Systems: Theories and Applications (SITA)*, 1–6. <https://doi.org/10.1109/SITA.2016.7772297>
7. Ahlam-Rauf, A., Wang, L.-Y. K., Bee, K.-S., & Thang, W.-R. (2023). **Architecture design of an IoT-based smart parking system.** *2023 International Conference on Future Internet of Things and Cloud (FiCloud)*, 136–141. <https://doi.org/10.1109/FiCloud58648.2023.00028>
8. Khalid, M., Wang, K., Aslam, N., Cao, Y., Ahmed, N., & Khan, M. K. (2021). **From Smart Parking Towards Autonomous Valet Parking: A Survey, Challenges and Future Works.** *Journal of Network and Computer Applications*, 175, 102935. <https://doi.org/10.1016/j.jnca.2020.102935>
9. **A Study on Smart Parking Solutions Using IoT.** (2019). *2019 International Conference on Intelligent Sustainable Systems (ICISS)*. <https://doi.org/10.1109/ISS1.2019.8907944>

## Software and Tools Used

1. **Python 3** – Used as the main programming language for developing the parking management system.
2. **Python Standard Library** – Modules such as datetime were used for recording vehicle entry and exit times.
3. **File Handling** – The system uses parking\_log.txt to store parking entry and exit transactions and to restore active parking sessions.
4. **Python Exception Handling** – try, except, and custom exceptions are used to handle invalid vehicle numbers, duplicate entries, unavailable slots and vehicles that are not parked. Python supports exception handling through its built-in exception mechanism.
5. **Console / Command-Line Interface** – The project uses a menu-driven console interface through which the operator can enter vehicles, process exits, check slot status, view tariffs and generate reports.

## Dataset Used

No external dataset was used in this project. The system works with **operator-entered vehicle registration numbers, vehicle types, parking slots, entry/exit times and transaction records**. The parking information is generated during program execution and stored in the parking log.

## Other Sources Used

The project was also referred to smart parking research to understand the general concept of parking allocation, parking management and smart parking systems. Fahim, Hasan and Chowdhury's review discusses different smart parking approaches, technologies, user interfaces and services.

## 16. Individual Reflection

Personal Review – S. Devadharshini

Working on the Bay-Watch Complex – Multi Level Parking Console Project knowing Python helped me to analytical how python concepts can be used to develop such a real time application. I knew what parking management was since the start of the project, I just needed to brainstorm on how vehicle entry, vehicle exit, billing, parking slot allotment, report generation, could function as one cohesive system.

On the menu, I also implemented the global flow of the program and the main operations of parking lot. I used the menu-driven technique to vehicle's entry and exit, the slot allocation and the different modules plugged together. Besides, I got to test the application like a vehicle entering a remote location, while entering and exiting the vehicles, for invalid plate numbers for repeating plates, for exiting plates that are not in the system.

One of the most important things I learned from this project was the value of modular programming. But rather than all the code being in one big long block, the responsibilities are split into modules: slot management, billing, transaction logging, and reports. That simplified this program a lot, and made it easier for other programmers to change this program.

Besides, I learnt practical, real-time examples and applications of discourse, dictionaries, sets, functions, tuples, file handling, exception handling and datetime module. The used parking log demonstrated to me that information can be saved and then retrieved when the program run again.

What I found out during testing is that a program can be right for normal input without any compensation for special cases of invalid input. This test with a diversity of cases enabled us to see and confirm how the system behaves. It also enhanced my critical thinking and debugging skills. Being in a team taught me how essential it is to split the work, talk about challenges, and review each other's coding. This project has given me more practical confidence in Python programming and has helped me realize how programming concepts can be integrated to create a practical application.